

## Concurrencia

- Introducción
- Opciones para la concurrencia
- Ejecutivo cíclico
- Programación concurrente
- Procesos en POSIX
- Hilos ("Threads") POSIX

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

1

## Esquema general

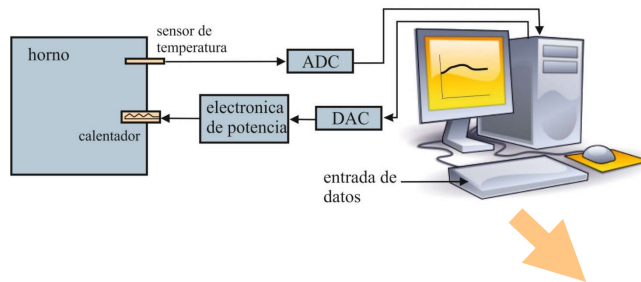
- Introducción
- Opciones para realizar la concurrencia
  - Programa único
  - S.O. monotarea con dos niveles de prioridad
  - Programación concurrente
- Ejecutivo cíclico
- Programación concurrente
  - Concepto, ventajas e inconvenientes
  - Diagrama de estados
  - Relaciones entre procesos
  - Modelos de concurrencia
- Procesos en POSIX
  - Detección de errores
  - Concurrencia en POSIX: procesos e hilos
  - Creación de nuevos procesos
  - Terminación y espera de procesos
- Hilos POSIX ("threads")
  - Concepto de hilo
  - Creación de un hilo
  - Terminación y espera de hilos
  - Compatibilidad hacia atrás y detección de errores

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

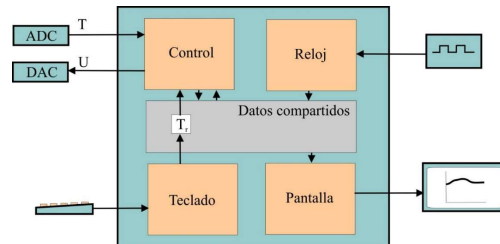
2

## Ejemplo de S.T.R



- Cuatro actividades concurrentes:

- Reloj de Tiempo Real
- Bucle de control
- Interfaz de usuario
- Representación gráfica

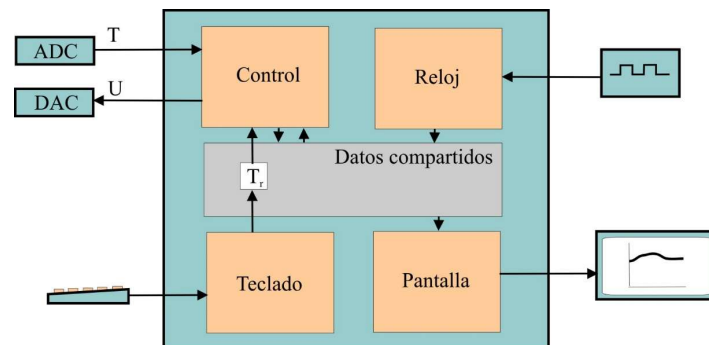


05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

3

## Ejemplo de S.T.R.

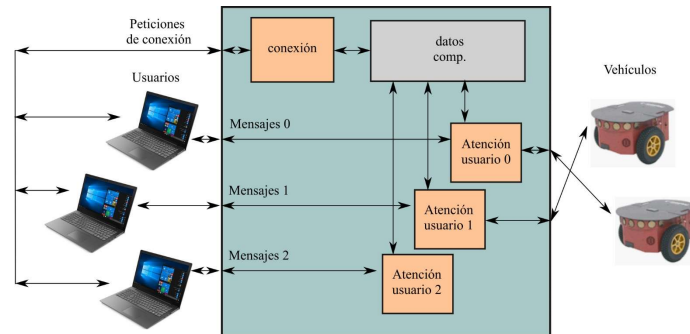


05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

4

## Ejemplo de S.T.R.



05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

5

## Soluciones para concurrencia en S.T.R.

- Más tareas que procesadores: Hay que **multiplexar en el tiempo el acceso**
- Diferentes soluciones para simular concurrencia:
  - Programa único (ejecutivo cíclico)
    - El programador reparte el tiempo de CPU entre las tareas
    - Las tareas son funciones invocadas por un programa único
  - Solución con dos niveles de prioridad:
    - Alta prioridad ("foreground"): **Tareas críticas**
    - Baja prioridad ("background"): **Tareas no críticas**
    - Hay conmutación a "foreground" cuando es necesaria
    - El programador reparte el tiempo de CPU en cada nivel
  - Sistema multitarea (programación concurrente):
    - Tareas definidas separadamente en programas independientes
    - Múltiples niveles de prioridad. El tiempo se asigna indirectamente mediante **prioridad** y algoritmos de **planificación**

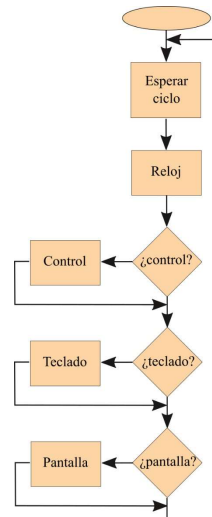
05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

6

## Soluciones para concurrencia en S.T.R.

- Solución de programa único (ejecutivo cíclico)
  - El programador reparte el tiempo de manera explícita
  - Los bloques son funciones de un programa
  - Comunicación directa a través de datos compartidos



05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

7

## Solución de programa único: Ejecutivo cíclico

- Todas las tareas son cíclicas
- Periodos múltiplos de uno básico: El **ciclo secundario** ("minor") – Normalmente el MCD de todos
- Varias tareas (funciones) se ejecutan consecutivamente en cada ciclo secundario
- Todas las tareas se repiten cada **ciclo principal** ("major") (MCM de todos los periodos)
- Sólo es adecuado para sistemas poco complejos

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

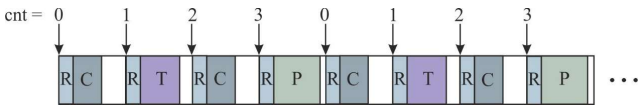
8

## Ejecutivo cíclico

- Ciclo secundario: 20 ms
- Ciclo principal: 80 ms (4 ciclos sec.)
- Plazo igual al periodo para todas
- Solución:
  - Ejecutar Reloj cada ciclo sec.
  - Ejecutar Control en ciclos pares
  - Ejecutar Teclado en ciclo 1
  - Ejecutar Pantalla en ciclo 3

| Tarea        | T  | C  |
|--------------|----|----|
| Reloj (R)    | 20 | 2  |
| Control (C)  | 40 | 8  |
| Teclado (T)  | 80 | 12 |
| Pantalla (P) | 80 | 14 |

| Ciclo | Tareas | Coste |
|-------|--------|-------|
| 0     | R,C    | 10    |
| 1     | R,T    | 14    |
| 2     | R,C    | 10    |
| 3     | R,P    | 16    |



05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

9

## Ejecutivo cíclico

### Pseudocódigo:

```
cnt = 0
Hacer siempre
  Esperar ciclo secundario (20 ms)
  Actualizar cuenta de tiempo (R)
  Si cnt es par, (cada 40 ms), Ejecutar C
    Leer entradas
    Recalcular salidas (PID)
    Actualizar registro de datos
    Actualizar salidas
  Si no,
    Si cnt es 1, (cada 80 ms)
      Si hay entrada del teclado de cambio de parámetros,
        Leerla y actualizar parámetros
      Fin si
    Si cnt es 3, (cada 80 ms)
      Actualizar gráficos de pantalla
    Fin si
  Fin si
  cnt = cnt + 1; Si cnt = 4, cnt = 0 /* Vuelta a empezar */
Fin hacer
```

### Más general, basado en tabla:

Hacer siempre  
Hacer para cada entrada cnt de la tabla,  
Esperar ciclo secundario (20 ms)  
Ejecutar tareas de la entrada cnt  
Fin hacer  
Fin hacer

| Ciclo | Tareas | Coste |
|-------|--------|-------|
| 0     | R,C    | 10    |
| 1     | R,T    | 14    |
| 2     | R,C    | 10    |
| 3     | R,P    | 16    |

05/10/2025

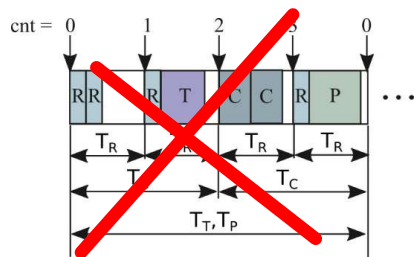
© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

10

## Ejecutivo cíclico

- Distribución adecuada de tareas:

- Se ejecutan **una sola vez** en cada uno de sus ciclos
- Terminan **antes del plazo** o "deadline" (da igual cuando comiencen exactamente dentro del ciclo si esto se cumple...)
- Si el plazo es igual al periodo, pueden ejecutarse en cualquier lugar de su ciclo
- En el ejemplo anterior, ¿valdría la solución para plazos iguales a la mitad del periodo?



### Distribución incorrecta:

- Sí, se ejecutan 4 "R", 2 "C", 1 "P" y 1 "T" en cada hiperperiodo,
- Pero:
  - En su primer periodo "R" se ejecuta **dos veces**, y en el tercero **ninguna**
  - En su primer periodo "C" **no se ejecuta**, y en el segundo **lo hace dos veces**

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

11

## Ejecutivo cíclico

- Problemas:

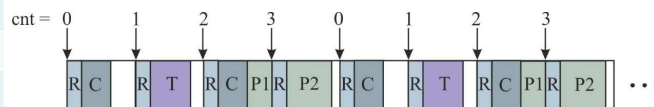
- Diseño potencialmente complejo
- Necesidad de rediseño ante modificaciones
- Poco adecuado para actividades esporádicas
- A veces es necesario fragmentar actividades

| Tarea        | T  | C  |
|--------------|----|----|
| Reloj (R)    | 20 | 2  |
| Control (C)  | 40 | 8  |
| Teclado (T)  | 80 | 12 |
| Pantalla (P) | 80 | 25 |

- ¿Qué hacer si Pantalla tarda más?

- Segmentarlo en P1 (10) y P2 (15) ☹

| Ciclo | Tareas | Coste |
|-------|--------|-------|
| 0     | R,C    | 10    |
| 1     | R,T    | 14    |
| 2     | R,C,P1 | 20    |
| 3     | R,P2   | 17    |



05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

12

## “Receta” para un ejecutivo cíclico

- Diseñar el ejecutivo:
  - Ciclo secundario (MCD de los periodos)
  - Ciclo principal (MCM de lo periodos)
  - Definir una secuencia adecuada de tareas
- Construir tabla
- Implementar el siguiente pseudocódigo:
 

```
Hacer siempre,
    Para cada entrada i de la tabla,
      Esperar el comienzo del ciclo secundario
      Ejecutar todas las funciones de la entrada i
    Fin Para
  Fin Hacer
```
- Cada iteración “Hacer” es un ciclo principal
- Cada iteración “Para” es un ciclo secundario
- Necesitamos una referencia básica de temporización

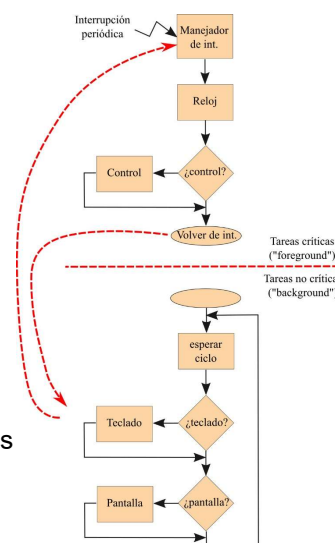
05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

13

## Soluciones para concurrencia en S.T.R.

- Solución con dos niveles de prioridad
  - Sist. operativo monotarea
  - Tareas críticas:
    - Invocadas por una rutina de servicio de interrupción
    - En el ejemplo:
      - Reloj de T.R.
      - Control
  - Tareas no críticas:
    - El programa “normal”, que es interrumpido por las tareas críticas
    - En el ejemplo:
      - Interfaz de usuario
      - Representación gráfica



05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

14

## Programación concurrente

- Tareas descritas como programas independientes
- El tiempo se asigna mediante algoritmos genéricos basados en prioridad (planificación – “scheduling”)
- Dos opciones:
  - A nivel de lenguaje: Lenguaje concurrente (ADA, JAVA...)
    - Todos los conceptos pertenecen al lenguaje: Programas más claros y mantenibles
    - El compilador puede detectar errores en la aplicación
    - Mayor portabilidad, que se supone resuelta por el compilador
    - Pueden no necesitar un sistema operativo completo para ejecutarse
  - A nivel de sistema operativo: Lenguaje secuencial + servicios s.o.
    - Necesario si hay que utilizar varios lenguajes
    - Implementación poco eficiente de lenguajes concurrentes sobre sistemas operativos
    - Los estándares permiten portabilidad

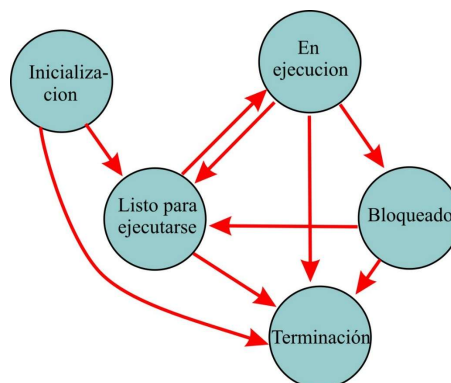
05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

15

## Programación concurrente

- Diagrama de estados para cada tarea
  - Las tareas listas para ejecutarse (“Ready”) piden CPU
  - Sólo una (o unas pocas) pueden ejecutarse en un instante dado
  - Las tareas bloqueadas no necesitan CPU; esperan algún evento
- Diagrama independiente del número de procesadores



05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

16

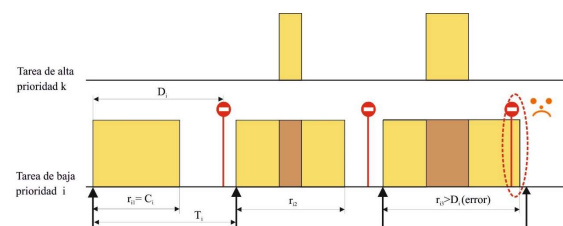


## Programación concurrente

- Planificación: actividad que toma decisiones sobre el estado de las tareas (cual se ejecuta en cada momento...)
- Habitualmente en sistemas operativos de tiempo real:

### Planificación basada en prioridades con expulsión ("priority-based preemptive scheduling")

- Las prioridades son normalmente fijas, y con significado claro
- La tarea más prioritaria pasa a ejecutarse lo antes posible si lo necesita, **expulsando** a las menos prioritarias



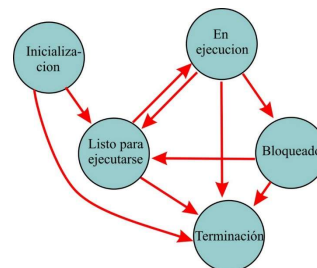
05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

17

## Programación concurrente

- Principales transiciones de estado:
  - Las tareas en ejecución pueden
    - Bloquearse, por necesitar esperar un evento
    - Ser expulsadas ("preempted") por una tarea más prioritaria y pasar a estar listas para ejecutarse
    - Terminar, dejando libre la CPU
  - Si deja de haber tarea en ejecución, se selecciona la tarea lista para ejecutarse de mayor prioridad para continuar su ejecución
  - Las tareas bloqueadas se desbloquean y pasan a estar listas para ejecución cuando:
    - Vence un temporizador
    - Llegan datos de entrada
    - ....
 ...y entonces pueden provocar una expulsión
- ¡Las tareas de alta prioridad **deben bloquearse**, o el resto no se ejecuta nunca!



05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

18

## Programación concurrente

- **Ventajas frente a otras soluciones:**
  - Modularidad
  - Facilidad de modificación
  - Escalabilidad
  - Portabilidad
  - Protección
- **Inconvenientes**
  - Menos eficiente que soluciones de propósito específico:
    - Mayores requerimientos de memoria
    - Mayores requerimientos de capacidad de computación
  - Son necesarios servicios de comunicación y sincronización
  - Es necesario estudiar la planificación de tareas mediante prioridades

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

19

## Modelos de concurrencia

- **Relaciones entre tareas**
  - Independencia
  - Cooperación
  - Competencia
- **Propiedades de los modelos de concurrencia**
  - Estructura: Estática o dinámica
    - Posibilidad o no de crear tareas en tiempo de ejecución (C+POSIX: dinámica)
  - Niveles de concurrencia: Plana o anidada
    - Posibilidad de descomponer tareas en subtareas concurrentes (C+POSIX: Dos niveles – hilos y procesos)
  - Granularidad: Nivel de descomposición en tareas
    - Granularidad "fina" para paralelismo masivo
    - C+POSIX: Normalmente relativamente pocas tareas y complejas
  - Inicialización: Padre e hijo
  - Terminación: Guardián y dependientes
- **Representación de tareas**
  - "Fork and join": C+POSIX
  - Declaración explícita: ADA

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

20

## Modelos de concurrencia

- “Fork and join”:
  - Concurrencia “fuera” del lenguaje
  - Creamos un objeto de s.o. y recibimos un identificador (“fork”: bifurcación)
  - Podemos esperar su fin (“join”: unión)
  - Es lo que sucede en C+POSIX (procesos e hilos)

```
...
c = fork(<tarea nueva>)
...
join(c);
```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

21

## Modelos de concurrencia

- Declaración explícita
  - Descripción integrada en el lenguaje
  - Las tareas se declaran como se hace con variables o funciones
  - ADA (“tasks”): La creación y espera de tareas puede estar implícita en el código
    - “ejemplo” arranca las tareas A y B al comenzar
    - “ejemplo” espera a A y B al acabar

```
procedure ejemplo is
  task A;
  task B;
  task body A is
    <declaraciones>
  begin
    <sentencias de A>
  end A;
  <lo mismo para B>
begin
  <sentencias de la
    “procedure” ejemplo>
end ejemplo;
```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

22

## Procesos en POSIX

- Detección de errores
- Concurrencia: Procesos e hilos
- Creación de nuevos procesos
- Terminación y espera de procesos

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

23

## Detección de errores (POSIX 1003.1a y 1003.1b)

- La llamada devuelve un código de error
  - Valor “anormal”, normalmente -1 (no siempre...)
  - En algunos casos, otros valores: NULL, EOF...
- Si hay error, el código de error está en **errno**
- Cabecera **<errno.h>**:
  - Declaración de **errno**
  - Definición de macros de códigos de error
- Para obtener un mensaje: **strerror** (en **<string.h>**)
- Algunos errores:
  - EAGAIN: Falta transitoria de recursos
  - EINTR: Función interrumpida por señal
  - EINVAL: Argumento no válido
  - ENOSPC: Medio de almacenamiento sin espacio

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

24

## Detección de errores

- Un ejemplo:

```
#include <stdio.h>
#include <errno.h>
#include <string.h>
(...)
int res;
res = una_llamada_posix(...);
/* Antes de usar errno, hay que comprobar si ha habido error */
if(res == -1) {
    /* Veamos cual */
    if(errno == EINTR) printf("¡Ha llegado una señal!\n");
    /* Mensaje explicativo, aunque no siempre útil */
    printf("Error: %s\n", strerror(errno));
}
(...)
```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y  
Automática, ESI Sevilla)

25

## Concurrencia en POSIX

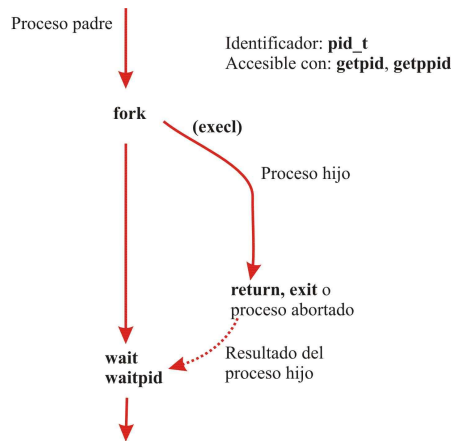
- Dos conceptos: Procesos e hilos
- Procesos:
  - Tienen espacios de direccionamiento virtual separados y su propia tabla de páginas
  - Tienen una entrada completa en la tabla de procesos
  - Se asegura la protección
  - No es inmediato compartir memoria
- Hilos:
  - Son siempre parte de un proceso; de algún modo son “subprocesos”, y el proceso es siempre un conjunto de hilos
  - Se necesita sólo un mínimo de recursos para crearlos:
    - Comparten la imagen de memoria del proceso
    - Comparten la mayor parte de los recursos del proceso; sólo se añade la mínima información necesaria a la tabla de procesos (p.ej. copia de los registros)
  - Ventajas:
    - Eficiencia
    - Comparten memoria automáticamente
  - Desventajas: No hay protección entre hilos

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y  
Automática, ESI Sevilla)

26

## “Fork and join” para procesos POSIX



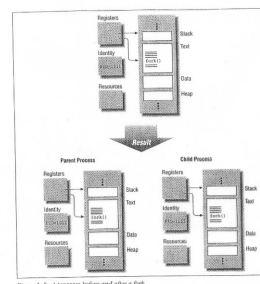
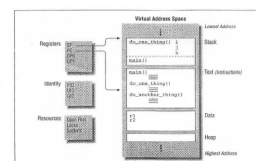
05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

27

## Creación de un proceso

- **Fork:**
  - Copia del padre
  - Idéntico estado de ejecución
  - Sólo un hilo; el que hizo **fork**
  - Distinto PID, distinta imagen de memoria
- Fork vuelve **dos veces** (si todo va bien):
  - En el padre: resultado **PID del hijo** ( $>0$ )
  - En el hijo: resultado **cero**
- El padre y el hijo comparten ejecutable. Si no interesa, necesitamos usar **exec**



05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

28

## Creación de un proceso

- **exec:**
  - Familia de funciones: **execl**, **execv**, **execle**...
  - Permiten superponer un ejecutable sobre la imagen de memoria de un proceso (destruyéndola)
  - Permiten pasar argumentos de línea de comandos
  - Segunda fase (opcional) de la creación de un proceso
- Una de ellas es **execl**:

```
int execl(const char *ejecutable, const char *arg0, ..., NULL);
```

- Si todo va bien, **nunca vuelve** (o “vuelve” al nuevo **main**)
- Transforma la imagen de memoria usando el nuevo ejecutable
- Pasa al nuevo **main** argumentos de la línea de comandos
- Sólo habrá un hilo que comienza en **main**
- Se mantienen algunos atributos (p.ej. máscara de señales)

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

29

## Creación de un proceso

- Sin exec:
 

```
(...)
pid_t hijo;
hijo = fork();
if(hijo == 0) { /* Estoy en el hijo */
    funcion_hijo(...); /* padre e hijo comparten ejecutable */
    exit(0);           /* exit termina el proceso hijo */
}
/* Aquí sigue el padre y nunca entra el hijo */
(...)
```
- Con exec:
 

```
(...)
pid_t hijo;
hijo = fork();
if(hijo == 0) { /* Estoy en el hijo */
    execl("hijo", "a", "b", NULL); /* hijo con ejecutable "hijo" */
    exit(0);                       /* (por si fracasa execl) */
}
/* Aquí sigue el padre y nunca entra el hijo */
(...)
```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

30

## Creación de un proceso

- Argumentos de la línea de comandos para **exec**:
  - Son siempre cadenas de caracteres: Si queremos enviar otro tipo de dato **hay que convertirlo a cadena de caracteres**
  - Una manera de convertir otros tipos (int, float...) a cadena de caracteres: Llamada **sprintf**
  - Habitualmente el primero (argumento 0) es el nombre del ejecutable (aunque en realidad podemos usar cualquier otro valor)
- Llamadas **printf**, **fprintf**, y **sprintf**:
  - **printf**: escribe con formato en la salida standard:  
`int printf(char *formato,...);`
  - Función **fprintf**: igual, pero escribe en un archivo:  
`int fprintf(FILE *fichero, char *formato,...);`
  - Función **sprintf**: igual, pero “escribe” en una cadena de caracteres:  
`int sprintf(char *input, char *formato,...);`

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

31

## Creación de un proceso

- Ejemplo con **sprintf**:
 

```
(...)
pid_t hijo;
char arg1[20]; /* Debe tener espacio suficiente */
float v1 = 4.5;
char *pv2 = "miarchivo.txt";
hijo = fork();
if(hijo == 0) { /* Estoy en el hijo */
    /* Espera algo que pueda convertirse a float en el argumento 1, y
    un nombre de archivo en el argumento 2 */
    sprintf(arg1, "%.2f", v1); /* v1 se convierte a cadena */
                                /* No es necesario hacer nada con pv2 */
    execl("hijo", "hijo", arg1, pv2, NULL); /* hijo con ejecutable "hijo" */
    exit(0); /* (por si fracasa execl) */
}
/* Aquí sigue el padre y nunca entra el hijo */
(...)
– El hijo recibe “hijo”, “4.50” y “miarchivo.txt” como args. de l.c.
```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

32



## Terminación y espera de un proceso

- **Terminación:**
  - Por iniciativa propia:
    - Con **return** desde **main**
    - Con **exit** o **\_exit** desde cualquier función
    - En ambos casos entrega un resultado (o “estado de terminación”) entero de 8 bits
  - Proceso abortado:
    - Causa: Normalmente la llegada de una señal que no se trata
    - No puede entregar realmente un resultado; lo hará el s.o.
  - El proceso queda pendiente de entregar resultado (“zombie”)
- **Espera de procesos hijos:**
  - La espera incluye la recogida del resultado
  - Sólo el padre puede esperar a un hijo, y debe hacerlo (si vive) para evitar dejar procesos “zombies”
  - Dos llamadas:
    - **wait:** Espera la terminación de cualquier hijo
    - **waitpid:** Puede esperar a un hijo concreto, o no esperar
  - El entero recibido **no** es directamente el resultado: Usar macros para recuperarlo: WIFEXITED, WEXITSTATUS

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

33

## Espera de procesos hijos

- **Con wait:**

```
(...)
pid_t resw, hijo; /* resultado de wait y pid del hijo */
int res;          /* resultado del hijo mas info. adicional */
hijo = fork();
if(hijo == 0) {
    funcion_hijo(...);
}
/* (el padre sigue ejecutándose)
resw = wait(&res); /* Espera (de cualquier hijo, pero en este caso sólo
                    hay uno) */
if(resw != -1) imprime_res(res, resw); /* Ya veremos qué contiene */
else printf("Error en wait: %s\n", strerror(errno));
(...)

```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

34

## Espera de procesos hijos

- Con **waitpid**:

```
(...)
pid_t resw, hijo; /* resultado de waitpid pid del hijo */
int res; /* resultado del hijo mas info. adicional */
hijo = fork();
if(hijo == 0) {
    funcion_hijo(...);
}
(el padre sigue ejecutándose)
do {
    salir = 1;
    resw = waitpid(hijo, &res, WNOHANG); /* Mira si el hijo ha acabado */
    if(resw > 0) imprime_res(res, resw); /* Ya veremos qué contiene */
    else if(resw == 0) {
        salir = 0;
        hacer_algo(); /* El padre aprovecha el tiempo... */
    } else printf("Error en waitpid: %s\n", strerror(errno));
} while(salir == 0);
```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y  
Automática, ESI Sevilla)

35

## Espera de procesos hijos

- Función **imprime\_res**:

```
void imprime_res(int result, pid_t id) {
    if(WIFEXITED(result)) {
        printf("%ld acabó normalmente con estado %d\n",
            (long)id, WEXITSTATUS(result));
    }
    else {
        if(WIFSIGNALED(result)) {
            printf("%ld muerto por señal\n", (long)id);
        } /* if */
        else {
            /* Otras posibilidades */
        } /* else */
    } /* if */
}
```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y  
Automática, ESI Sevilla)

36

## Hilos (“threads”)

- Concepto de hilo
- Creación de un hilo
- Terminación y espera de hilos
- Compatibilidad hacia atrás y detección de errores

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

37

## Concepto de hilo

- Todo hilo está asociado a un proceso
- Hilos de un proceso (“threads”):
  - Comparten espacio de direcciones (usan la misma tabla de páginas)
    - Por tanto, variables globales accesibles desde todos ellos
  - Recursos compartidos (p. ej. ficheros)
  - No se duplican todos los datos de la tabla de procesos, sólo se añade lo imprescindible
  - No se duplica la imagen de memoria: Sólo se crea una **pila nueva** en ella
- **Ventajas:**
  - Creación menos costosa que proceso
  - Conmutación menos costosa (**ojo**: entre hilos del mismo proceso)
  - Facilidad para compartir memoria
- **Desventaja:** No existe protección entre hilos

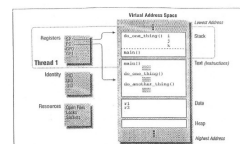


Figure 2-2: The simple program as a process with a thread.

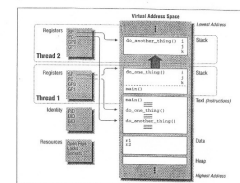


Figure 2-3: The simple program as a process with multiple threads.

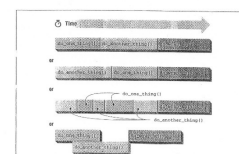


Figure 2-4: Possible sequences of the execution in the simple program.

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

38

## Concepto de hilo

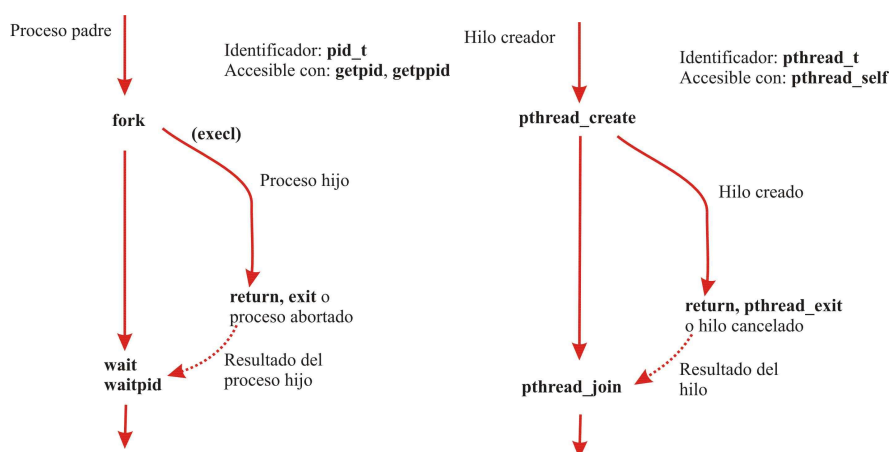
- Concepto existente en la mayor parte de los sistemas operativos (también llamados “procesos ligeros” o “lightweight processes”)
- Los hilos POSIX suelen llamarse “`threads`”
- ¿Cuándo usarlos?
  - Funciones estrechamente relacionadas
  - Dedicar hilos a tratar los diferentes periféricos: así los que necesiten cálculo intensivo no se bloquearán por E/S
  - Eficiencia: Utilizando sólo hilos en lugar de múltiples procesos conseguimos eficiencia a cambio de falta de protección
- Algunos sistemas operativos pueden opcionalmente trabajar sólo con hilos (“thread-only”); por ejemplo, VxWorks – Tornado
  - Prescinden del modo de direccionamiento virtual y paginado
  - Anulando este modo de direccionamiento el procesador suele ir más rápido
  - Mayor eficiencia (por varios motivos), menor protección

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

39

## “Fork and join” para procesos e hilos



05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

40

## Creación de hilos

- Similar a una llamada a función, pero ésta funciona concurrentemente con el resto: **Función de arranque**
- Llamada **pthread\_create**:
 

```
int pthread_create(pthread_t *h, const pthread_attr_t *attr,
                  void *(*rut_com)(void *), void *arg);
```

  - Similar a ejecutar **rut\_com(arg)**
  - Función de arranque: Acepta un puntero genérico y devuelve un puntero genérico
  - Una función de arranque: `void *rhilo(void *p);`
  - El dato apuntado por **arg** **no puede ser volátil**
    - Suele ser mala idea pasar un puntero a una variable automática...
  - La estructura **attr** se utiliza para modificar los atributos del hilo (prioridad, dirección y tamaño de la pila...)
  - **pthread\_t** no tiene por qué ser un entero
  - No hay relación padre-hijo más allá de la creación del hilo

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

41

## Creación de hilos

- Creación de un hilo:

```
(...)
void *mirut(void *pg); /* Prototipo de función de arranque */
(...)
pthread_t idh;
int *p; p = malloc(2*sizeof(int)); /* mirut espera una tabla de 2 enteros
                                   reservada con malloc */
pthread_create(&idh, NULL, mirut, (void *)p); /* Ahora el hilo va en paralelo */
(...)
/* Función de arranque */
void *mirut(void *pg) {
    int *pt = (int *)pg;
    (usa pt[0] y pt[1])
    (...)
    free(pg); /* Hay que liberar pg cuando ya no sirva */
    return NULL; /* Fin de mirut y del hilo */
} /* Podría haber devuelto un puntero válido,
   igual que recibió el argumento */
```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

42

## Terminación y espera de hilos

- **Terminación:**
  - Por iniciativa propia:
    - Con **return** desde la función de arranque
    - Con **pthread\_exit** desde cualquier función
    - En cualquier caso se devuelve un **puntero genérico** como resultado; el dato al que apunta **no debe ser volátil** (!no devolver un puntero a una variable automática!)
  - Por cancelación:
    - Otro hilo emite una orden de cancelación con **pthread\_cancel**
    - El hilo **no** puede devolver realmente un resultado (lo hace el s.o.)
  - Normalmente el hilo espera a que recojan el resultado (“zombie”)
- **Espera:**
  - Cualquier hilo puede esperar a otro con **pthread\_join**, y alguno debería hacerlo (salvo si se usa **pthread\_detach**):
 

```
int pthread_join(pthread_t thread, void **valor);
```
  - Si el hilo fue cancelado, se recibe PTHREAD\_CANCELED (un puntero ilegal), en vez del resultado que no ha podido enviar
  - Si se ha usado **pthread\_detach** con un hilo, éste no queda pendiente de entregar resultado (no hay hilo “zombie”), y no puede usarse con él **pthread\_join** (devuelve error)

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

43

## Terminación y espera de hilos

- **Creación y espera de la terminación de un hilo:**

```
(...)
void *mirut(void *pg); /* Prototipo de función de arranque */
(...)
pthread_t idh; int *result; void *paux;
int *p; p = (int *)malloc(2*sizeof(int)); /* mirut espera una tabla de 2 enteros
                                          reservada con malloc */
pthread_create(&idh, NULL, mirut, (void *)p); /* Ahora el hilo va en paralelo */
(...)
pthread_join(idh, &paux); result = (int *)paux; /* Esperamos un puntero a entero */
printf("result %d\n", *result); free(paux); /* paux debe devolverse si no se usa */
(...)
/* Función de arranque */
void *mirut(void *pg) {
    int *mires;
    (todo como antes)
    mires = (int *)malloc(sizeof(int)); *mires = 18; /* Ahora sí devolvemos algo */
    return (void *)mires; /* O bien pthread_exit((void *)mires); */
}
```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

44

## Hilos: Argumentos y resultados

- Los punteros deben tener sentido cuando se utilizan:
  - Argumento válido mientras dure el hilo
  - Resultado válido cuando acaba el hilo
  - No suele tener sentido que hilos distintos reciban el mismo argumento
- No siempre es imprescindible usar **malloc**; puede haber otras opciones:
  - Puntero a variable local de **main**
  - Puntero a variable global
  - Un elemento de tabla como argumento para cada hilo
- Para pasar varios valores de diferente tipo: Estructura
- Un “truco” si sólo se pasa un entero:
  - Convertir el entero a puntero (argumento o resultado) para pasarlo, y de puntero a entero para usarlo
  - No es lo más correcto, pero normalmente funciona
  - En resultado, el entero puede confundirse con `PTHREAD_CANCELED`
  - No puede convertirse cualquier tipo a puntero, sólo escalares y del mismo o menor tamaño que un `void *` (tiene que “caber”...)

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

45

## Hilos: Argumentos y resultados

- Cómo pasar un entero como puntero:

```
(...)
void *mirut(void *pg); /* Prototipo de función de arranque */
(...)
pthread_t idh; int result; void *paux; int arg = 7;
pthread_create(&idh, NULL, mirut, (void *)arg);
(...)
pthread_join(idh, &paux); result = (int)paux; /* Esperamos un entero */
/* Ojo: Si el hilo fuese cancelado, tal vez no podríamos detectarlo, porque
   PTHREAD_CANCELED es un entero válido... */
printf("result %d\n", result);
(...)
/* Función de arranque */
void *mirut(void *pg) {
    int miarg, mires;
    miarg = (int)pg;          /* El puntero se convierte a entero */
    (...)
    mires = 18;
    return (void *)mires; } /* El entero se devuelve como puntero */
```

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

46

## Cancelación de hilos

- Es similar a abortar un proceso enviándole una señal
- Cualquier hilo puede intentar cancelar a otro con **pthread\_cancel**
- La cancelación depende de:
  - Estado de cancelación: Habilitada o deshabilitada
  - Tipo de cancelación:
    - **Asíncrona**: Lo antes posible (puede ser poco recomendable)
    - **Síncrona** (“deferred”): En puntos de cancelación explícitos (**pthread\_testcancel**) o implícitos (la mayor parte de las llamadas bloqueantes, que implican esperas)
- Un hilo puede alterar tanto su estado como su tipo de cancelación
  - Llamadas **pthread\_setcancelstate** (estado) y **pthread\_setcanceltype** (tipo)
- Por defecto: **Habilitada y síncrona**

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

47

## Hilos: Compatibilidad con normas anteriores

- El concepto de hilo no existía originalmente en POSIX
  - Problemas para mantener la compatibilidad “hacia atrás”
- Procesos e hilos
  - Un hilo pertenece siempre a un proceso
  - En realidad la “unidad de concurrencia” **es el hilo**, no el proceso
  - Un proceso siempre tiene al menos un hilo que normalmente comienza en **main**
  - Después de **fork** el hijo sólo tiene un hilo (el que ha ejecutado **fork**)
    - En QNX 6.2.3 esto es irrelevante, no puede hacerse **fork** después de crear el primer hilo adicional a **main**...
  - El hilo “**main**” es “especial”:
    - No es creado con **pthread\_create**; aparece tras ejecutar **exec**
    - No comienza en una función de arranque tipo “void \*(void \*)”
    - Si acaba con **return** o fin del código, **acaba el proceso completo** (todos sus hilos)
    - No obstante si **main** acaba con **pthread\_exit** el resto de hilos no se ve afectado, y el proceso sigue existiendo

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

48



## Hilos: Compatibilidad con normas anteriores

- **Notificación de errores:**
  - Las llamadas “antiguas” siguen usando **errno**
  - Errores de diferentes hilos podrían confundirse, por tanto
    - Hay una variable indicadora del error por hilo
    - El nombre **errno** es un macro, y hace referencia a la que corresponde al hilo actual
  - Las llamadas **pthread\_xx** no tienen por qué usar **errno**; Devuelven el código de error como resultado (0 es “no error”)
- **Llamadas seguras en hilos (“thread-safe”):**
  - Sólo las llamadas “thread-safe” pueden invocarse concurrentemente desde hilos distintos
  - Llamadas no seguras:
    - Habitualmente utilizan alguna variable global
    - Pueden utilizarse si no se invocan de manera concurrente
    - Podrían ser seguras asegurando internamente el acceso exclusivo a datos globales
  - La mayoría de las llamadas 1003.1a y 1003.1b son seguras en hilos

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

49

## Procesos e hilos: Puntos a recordar

- **fork:**
  - Tras **fork** ya hay un proceso hijo, (casi) idéntico al padre
  - Padre e hijo harán exactamente lo mismo si no hacemos nada
- **Argumentos de la línea de comandos (a.l.c.) y main:**
  - **main** tiene sólo dos argumentos (**argc** y **argv**, por ejemplo) no todos los que queramos
  - Los a.l.c. son siempre cadenas de caracteres
  - Si se interpretan como cualquier otra cosa hay que hacer una conversión (**sscanf**, por ejemplo)
  - El primer a.l.c. (el 0) suele ser el nombre del ejecutable, y por tanto no suele ser útil
  - Al darles valor en **exec**, también necesitaremos una conversión (**sprintf**, por ejemplo) si pasamos algo que no es una cadena de caracteres
- **Terminación y espera:**
  - La terminación del padre no provoca la terminación de los hijos
  - Si el hijo no ha terminado, el padre espera en **wait** o **waitpid**
  - Si ya lo ha hecho, **wait** y **waitpid** no esperan; sólo devuelven el resultado (es el hijo el que ha “esperado” en estado “zombie”)

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

50

## Procesos e hilos: Puntos a recordar

- **Concepto de hilo:**
  - Un proceso es un conjunto de hilos que comparten recursos
  - **main** es sólo un hilo más, un poco especial; no es “el proceso”
  - Si **main** acaba (salvo con **pthread\_exit**), **todos** los hilos del proceso acaban - luego **más vale que tenga algo que hacer...**
  - Todos los hilos pueden usar los recursos de su proceso (otra cosa es que tenga sentido cómo lo hacen...)
  - **pthread\_detach** no termina un hilo; sólo evita que se quede “zombie” al acabar
- **Argumentos y resultado de un hilo:**
  - Un hilo (salvo **main**) **sólo tiene un argumento** (un void \*), no todos los que queramos
  - Si hay que pasar por argumento varias variables, podemos utilizar una estructura o una tabla (si son del mismo tipo)
  - Si pasamos como argumento un puntero idéntico a varios hilos, **todos acceden la misma variable** (mala idea, normalmente)
  - El puntero que recibe un hilo debe tener sentido al menos mientras el hilo lo utilice
  - El puntero que devuelve un hilo debe tener sentido cuando el hilo acaba

05/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

51